# Workload-Shaped Checkpoint Loading for Large Language Model Inference

**Botir Khaltaev**
Independent, United Kingdom
btrghstk@gmail.com

## Abstract

LLM cold-start latency is dominated by reading checkpoint weights from storage into GPU memory. Peak NVMe bandwidth is only an upper bound: realized throughput depends on the I/O request geometry determined by checkpoint layout, the submission policy of the loading backend, and integration overhead from the serving stack. This paper shows that checkpoint layout determines request geometry, which changes the optimal loading backend across scales and integration layers.

Using Tensora, an open checkpoint-loading measurement framework, we evaluate synchronous POSIX I/O, Tokio-based async, io_uring, and an adaptive default policy across two layouts (SafeTensors contiguous shards and ServerlessLLM-style partitioned files) at three layers: a Rust microbenchmark for pure backend attribution (six public checkpoints from 0.6B to 32B parameters), Python bindings for user-facing overhead, and a custom vLLM integration for end-to-end serving impact.

Three findings emerge. First, SafeTensors exhibits a clear crossover: synchronous loading is fastest for small models, while io_uring dominates for multi-shard checkpoints at 8B parameters and above; no fixed backend tracks both regimes. Second, the partitioned ServerlessLLM layout reverses the ranking—synchronous loading never wins, and the relevant trade-off is between async and io_uring depending on partition structure. Third, Python tensor materialization and vLLM initialization overhead change which backend appears fastest, but adaptive policy consistently outperforms any single fixed choice. In vLLM cold start, Tensora's adaptive SafeTensors path reduces initialization time by 37–48% compared to the native loader for 8B and 32B models.

The operational conclusion is not a single universal loader. It is a format-aware adaptive policy with explicit overrides, grounded in measured workload features—bytes, shard count, partition count, and integration layer—rather than API names alone.

***Keywords*** large language models · checkpoint loading · I/O policy · io_uring · cold start · vLLM · reproducibility

## 1 Introduction

An LLM serving request cannot begin until weights reach GPU memory. A 7B parameter model in float16 requires roughly 14 GB; serverless and multi-tenant environments evict idle models to reclaim memory, so every new request may trigger a cold load. This cold-start latency is directly visible to users, and its dominant component is I/O: reading checkpoint bytes from storage.

Peak NVMe bandwidth is only an upper bound. Checkpoint layout determines whether reads are contiguous or fragmented; the I/O backend determines how those reads are submitted, batched, and completed; Python bindings and serving engines add integration overhead that can mask backend-level differences. The gap between peak hardware and realized loading throughput is a systems problem in how reads are scheduled, not just how fast the storage device can transfer data.

**Thesis.** Backend policy for LLM checkpoint loading cannot be divorced from checkpoint layout or integration layer. A single fixed backend—always synchronous, always async, always io_uring—is structurally suboptimal because two layouts produce fundamentally different I/O geometries: contiguous shard loads that benefit from bulk parallelism, and partitioned range loads dominated by fragmentation and grouping. Scale further shifts the optimum within each layout.

**Evidence.** Using Tensora[1], an open checkpoint-loading measurement framework, we evaluate four backend policies (synchronous POSIX I/O, Tokio-based async, io_uring, and an adaptive default) across two formats (SafeTensors contiguous multi-shard and ServerlessLLM-style partitioned layouts) at three layers (Rust microbenchmark, Python bindings, vLLM integration). The SafeTensors Rust matrix exhibits a clear crossover: synchronous loading is fastest at small scales (0.6B–4B parameters), while io_uring dominates from 8B parameters upward. For ServerlessLLM, synchronous loading is never the winner. Python tensor materialization adds enough overhead to reorder backends; vLLM initialization amplifies loader effects by 37–48% relative to the native loader at 8B and 32B scales.

**Contributions.**

1. A workload-shape characterization of LLM checkpoint loading showing that format determines I/O request geometry, which changes the optimal backend policy.
2. A controlled measurement study comparing four backend policies across two checkpoint layouts, producing regime maps showing format-dependent crossovers.
3. Cross-layer integration evidence from Python bindings and vLLM showing that backend rankings change across layers and that adaptive policy survives this reordering.
4. An operational framework: deploy a format-aware adaptive policy with explicit overrides, grounded in measured workload features rather than API names.

**Roadmap.** Section 2 positions the gap. Section 3 describes Tensora and the adaptive policy. Section 4 defines methodology. Section 5 presents results across three layers. Section 6 interprets findings, Section 7 bounds claims, and Section 8 summarizes.

## 2 Related Work

This section positions our thesis—that LLM checkpoint loading performance depends on workload shape, not just peak bandwidth, and that backend policy must be format-aware and adaptive—against existing work. We cover serving systems, serverless cold-start solutions, checkpoint formats and loading systems, and I/O backend research. The common thread: prior work optimizes around the load, rather than measuring and adapting the load itself.

### 2.1 LLM Serving Systems

Production LLM serving engines optimize inference throughput and latency once weights are in GPU memory. vLLM introduces PagedAttention for efficient KV-cache management [1]. Orca pioneers iteration-level batching [2]. Sarathi-Serve refines the prefill-decode scheduling trade-off [3]. DeepSpeed Inference applies model parallelism and kernel fusion [4]. These systems treat checkpoint loading as a one-time fixed cost before their pipeline starts. Faster loading directly reduces time-to-first-token, which these systems treat as an externally determined constant.

Our work fills this gap by measuring and optimizing the loading phase that serving engines assume away.

### 2.2 Serverless Cold Start

Serverless LLM serving introduces the cold-start problem: idle models are evicted and must be reloaded on demand. ServerlessLLM [5] proposes multi-tier loading with GPU/NVMe/DRAM placement and live migration to reduce this cost. HydraServe [6] minimizes cold-start latency in public clouds through pipeline parallelism and placement optimization. Snapshot-based warm-up approaches [7] pre-materialize model state to bypass loading altogether.

[1]Tensora: this paper's checkpoint-loading framework (Rust crate `tensora`, Python package `tensora_py`). It is unrelated to Google's TensorStore library for array storage.

All of these optimize *whether* and *when* loading occurs, or structure the storage hierarchy to reduce its frequency. None measure the I/O mechanics of the load itself—the submission policy, request geometry, and backend selection that determine how fast bytes move once loading is unavoidable. Our work holds scheduling fixed and measures these load mechanics across backends, formats, and integration layers.

### 2.3 Checkpoint Formats and Loading Systems

Checkpoint formats determine what byte ranges a loader must read and whether access is contiguous or fragmented. SafeTensors [8] provides safe, fast tensor serialization with a contiguous multi-shard layout. GGUF optimizes for memory-mapped CPU inference at the cost of quantized weights. ServerlessLLM's partitioned layout [9] distributes tensors across multiple files with a central index, enabling partial loading and tiered storage placement.

Recent systems work optimizes the loading path from different angles. BlitzScale [10] uses O(1) GPU-side model caching over NVLink and RDMA to share already-loaded models between inference instances. Most relevant to our work, PPC/MAIO [11] optimizes model loading through a programmable kernel page cache, achieving substantial speedups by adapting prefetching and eviction policies to LLM I/O patterns. MAIO operates below the system-call interface; our work operates above it, comparing userspace I/O backends on identical logical read plans. The two approaches are complementary: MAIO optimizes how the kernel serves pages, while Tensora optimizes how the application submits reads.

A separate line of characterization work studies LLM checkpoint/restore I/O patterns on parallel file systems [12], covering buffered versus direct I/O paths and liburing performance. That work focuses on training checkpointing in HPC environments; our focus is inference checkpoint loading in single-node serving.

The gap we address: given a fixed format, which I/O backend submits reads fastest, and how does the answer change with scale, layout, and integration layer?

### 2.4 I/O Backend Policy

The Linux I/O interface landscape spans synchronous POSIX, POSIX AIO, and io_uring. Recent database research provides the closest methodological template for our work: Jasny et al. [13] show that io_uring benefits depend on workload shape and queue depth, with bulk sequential reads already amortizing syscall overhead while fragmented many-small workloads benefit from batched submission. This is directly relevant because checkpoint loading spans both regimes: contiguous SafeTensors shards resemble bulk sequential access, while partitioned ServerlessLLM layouts resemble fragmented range workloads.

Our work translates these database findings to a new domain (LLM checkpoint loading) and extends them with cross-layer evidence (Rust, Python, vLLM) and an adaptive policy grounded in workload features rather than API names.

### 2.5 Positioning

We do not introduce a new checkpoint format, optimize serving throughput, or solve scheduling. We measure load mechanics across four backend policies and two layouts, for six public checkpoints, at three integration layers. The contribution is operational: evidence that backend choice depends jointly on layout and scale, and a format-aware adaptive policy with explicit overrides. This is the missing piece between format design (what bytes exist), serving engines (how they use loaded weights), and kernel-level cache optimization (how the page cache serves reads).

## 3 Tensora Design

This section describes the design of *Tensora*, the experimental framework we built to conduct this measurement study. The system is the vehicle for our evaluation, not the contribution itself.

### 3.1 Design Goals

Tensora was designed around four primary requirements. First, the framework must support multiple I/O backends and checkpoint formats under a consistent interface, so that backend decisions can be studied without rewriting the rest of the loading stack. Second, the framework must keep the loader logic separate

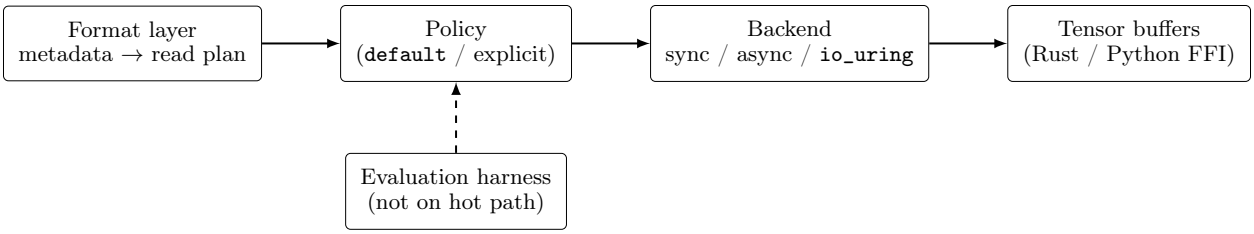


Figure 1: Tensora architecture: the format layer produces a normalised read plan; policy selects a backend and planning parameters; backends execute I/O; evaluation harnesses attach at the boundary for measurement only.

from the measurement harness, so that instrumentation does not distort the measured path. Third, the design must make its concurrency model explicit, because queue depth, worker fanout, and batching policy are central to the performance question. Fourth, the framework must remain accessible from Python and from a real serving engine, so that backend-level findings can be tested beyond Rust-only microbenchmarks.

These requirements led to a layered architecture that separates format parsing, I/O execution, policy selection, and evaluation.

### 3.2 System Architecture

Tensora is structured into three layers: a format layer, a backend layer, and an evaluation layer.

The format layer parses checkpoint metadata and produces a normalised loading workload. The key abstraction is a request-local view of the checkpoint: the format layer determines which file ranges must be read, but it does not commit to how those reads are scheduled. This separation allows the same logical workload to be executed by different backends while preserving the semantics visible to the caller.

The backend layer implements I/O execution. Each backend receives the same logical workload and executes it according to its own submission model. Backends may perform backend-local planning such as chunking large files or coalescing nearby ranges, but they must preserve request-local output semantics. That matches how real loaders behave: the same logical byte ranges can be fetched with different scheduling while preserving tensor outputs, and comparing backends fairly requires allowing each implementation its natural planning.

The evaluation layer contains the Rust profiling harness, the Python benchmarks, and the vLLM integration benchmarks. These are kept outside the core loading path so that the production implementation and the measurement logic remain distinct.

Figure 1 summarises the layering; the following subsections specify the backend interface, adaptive policy, and bindings.

### 3.3 Common Backend Interface

All eager-loading backends implement the same public surface:

- `load(path)` for a full file,
- `load_batch(paths)` for multiple whole-file loads,
- `load_range(path, offset, len)` for one byte range, and
- `load_range_batch(requests)` for many ranges.

This interface is deliberately narrow: it covers `SafeTensors` and `ServerlessLLM` without extra surface area, so backend differences remain interpretable.

### 3.4 Workload Features

The format layer extracts a common set of workload features from every checkpoint, regardless of layout. These features drive both the analysis and the adaptive policy:

- **Total bytes**: the aggregate checkpoint size across all shards or partitions.

- **Shard count**: number of discrete files (SafeTensors) or partitions (ServerlessLLM).
- **Request count**: number of byte-range reads required to load all tensors.
- **Contiguity ratio**: fraction of total bytes addressable as sequential ranges versus fragmented reads.
- **Format family**: SafeTensors (contiguous multi-shard) or ServerlessLLM (partitioned range-oriented).

SafeTensors checkpoints are characterized by low request count and high contiguity—tensors are stored as whole-file contiguous chunks. ServerlessLLM checkpoints have high request count and low contiguity—each tensor may span multiple partition files, producing many fragmented range reads. These differences are the root cause of the backend ordering reversals observed in Section 5.

### 3.5 Adaptive Loading Policy

The central design decision is that backend selection is handled as an adaptive heuristic rather than as a fixed policy. The Rust matrix (Tables 3 and 4) shows that no single backend is best across both layouts and all scales.

The `default` policy operates in three stages. First, `extract_features` reads workload signals from the format layer: total bytes, shard count, partition count, and contiguity features. Second, `heuristic_select` maps features to a backend: sync for low-request-count, high-contiguity loads below the crossover threshold; io_uring for high-request-count or large-scale loads; async for mid-range partitioned loads where grouped range reads fit its execution model. Third, `plan_reads` applies backend-specific planning (chunking large files, coalescing nearby ranges, setting in-flight depth) before execution.

This means `default` is not a pure selector among the explicit benchmark paths. It combines adaptive backend selection with execution-time planning decisions absent from the explicit columns. This design is intentional: in production, a loader should both choose the right backend *and* execute it well, and comparing default against explicit backends separates the benefit of backend selection from the benefit of planning.

For `SafeTensors`, the heuristic maps shard count and total bytes to a regime decision: **synchronous execution for smaller eager loads**, **io_uring for larger multi-shard loads**. For `ServerlessLLM`, the features are partition count and request count; the heuristic does *not* route to synchronous I/O (the Rust matrix shows sync never wins on this layout). Smaller and mid-sized range-heavy workloads may use the Tokio async path; larger partitioned checkpoints route to `io_uring`.

The selector is **feature-driven**, not a single hard-coded byte threshold. Exact internal cut-offs are omitted here to avoid false precision; regimes matter more than constants. Figure 2 summarises control flow; threshold constants live in the open-source implementation alongside the harness revision recorded for results. Table 5 in Section 5 quantifies the selector's boundedness: it tracks the best explicit backend in 9 of 12 Rust-layer cells and deviates most where the medium-scale crossover region makes regime boundaries ambiguous.

```
extract_features
       ↓
heuristic_select
       ↓
   plan_reads
       ↓
    execute
```

Figure 2: Adaptive policy control flow: features → backend selection → read planning → execution. SafeTensors uses sync for small and io_uring for large loads; ServerlessLLM routes to async or io_uring depending on partition structure.

### 3.6 Backend Designs

The synchronous backend uses blocking POSIX reads. In the final design it is not a purely serial implementation: large files are split into chunks and loaded with host-side thread parallelism. This keeps the backend simple at the API boundary while still exploiting multiple cores where appropriate. The design intent is not to make `sync` look trivial, but to give it a fair and practical implementation baseline.

Table 1: Major design decisions and rationale.

| Decision | Alternatives considered | Rationale |
|---|---|---|
| Adaptive `default` | Fixed global backend | Evidence shows crossovers with scale and layout |
| `io_uring` implementation | Minimal parallelism / naive batching | Worker-owned rings with chunking and planning sustain queue depth on large multi-shard loads |
| Cold-cache control | In-process eviction hints | External `sync`/`drop_caches` separates harness from loader |
| vLLM integration | Async loader bridge | vLLM's loader API at the Python boundary is synchronous; benchmarks follow that interface |
| Python Tokio | New runtime per call | Shared runtime reduced binding overhead |

The async backend is built on Tokio. Its design is strongest for grouped and range-heavy workloads rather than for large whole-file eager loads. The final implementation groups work per file, uses dynamic batching, and reduces unnecessary task fanout. This backend remains important because the final results show that explicit async loading is often strong for `ServerlessLLM`-style partitioned workloads even when it is weak for `SafeTensors`.

The `io_uring` backend uses multiple worker-owned rings, lazy ring initialisation, dynamic ring sizing, and workload-driven planning for chunk size and in-flight depth, so that submission can keep pace with large multi-shard and range-heavy workloads.

Table 2: Evaluated `io_uring` backend structure.

| Aspect | Design |
|---|---|
| Ring ownership | Worker-owned rings (lazy initialisation) |
| Submission | Multiple workers; batched SQEs; depth matched to workload |
| Planning | Dynamic ring sizing, chunk sizing, and in-flight limits |

We do not conclude that `io_uring` is inherently superior. We conclude that `io_uring` pays off when the implementation sustains enough parallelism for the workload; topology and planning matter as much as choosing the API family.

The mmap-backed path is retained as a useful lazy-access mode, especially for Python and vLLM experiments where a caller may not need to materialise every tensor immediately. It is not treated as the main eager-loading solution to the cold-start problem, because mmap solves a different problem: efficient selective access rather than aggressive eager loading.

### 3.7 Checkpoint Formats

Tensora supports two checkpoint layouts.

`SafeTensors` stores tensors in a whole-file or multi-shard layout. Once the header is parsed, the loader sees a largely contiguous eager-loading workload.

The `ServerlessLLM` layout distributes tensors across multiple partition files with a central index. This introduces a more fragmented range-oriented workload and therefore stresses a different part of the backend design space.

The purpose of supporting both formats is not just compatibility. It is to expose two different I/O regimes under one framework.

### 3.8 Python Bindings

Tensora exposes its functionality through Python bindings implemented with PyO3. The bindings intentionally expose a small public API: a `default` eager path, explicit `sync` and `async` eager paths, and lazy `open_*` mmap-backed handles. On Linux, `io_uring` remains an internal backend behind `default` in the public Python API, while benchmark-only explicit paths were used in the vLLM benchmark layer.

One important binding design decision is that the async Python path runs on a shared Tokio runtime rather than constructing a fresh runtime per call. This was one of the practical optimisations that materially improved Python-side performance.

The vLLM integration is implemented through a custom model-loader path. vLLM is particularly useful as an evaluation target because it separates model initialisation, time-to-first-token, and steady-state decode. This allows us to test whether backend gains survive contact with a real serving engine.

The key design constraint is that vLLM's loader interface is synchronous. The final benchmark integration therefore compares synchronous and default eager-loading policies directly, and uses benchmark-only explicit loader variants where necessary to expose additional backend behaviour. This is a practical design choice: it reflects the serving interface that actually exists, rather than inventing an async serving path that the engine does not support.

Cold-cache control is implemented in the experiment harness, not inside the loader: the Linux shell runs `sync` and `drop_caches` before benchmark invocations. That keeps measurement ritual separate from loading logic and matches common storage-evaluation practice.

This decision is important for clarity. The loader implementation should express how Tensora loads checkpoints; the benchmark harness should express how cold and warm conditions are created. Tables 1 and 2 summarise principal design choices; the narrative above explains Rust core choice, shared interface, and binding integration in full.

## 4 Experimental Setup

This section describes the experimental methodology used to evaluate the checkpoint loading backends. We make explicit what each measurement layer captures and what it does not capture, so that readers can correctly interpret which claims are supported by which evidence.

### 4.1 What is measured and why the separation matters

The evaluation spans three distinct measurement layers, each answering a different question:

1. **Rust-layer microbenchmark.** This measures pure I/O and format deserialisation: reading checkpoint bytes from storage through the selected backend and assembling tensors in memory. It intentionally excludes Python interpreter overhead, tensor materialisation in other runtimes, and GPU allocation inside a serving engine. The purpose is backend attribution - isolating which submission policy (sync, async, io_uring, default) produces the best raw loading performance for a given checkpoint layout.
2. **Python-layer binding benchmark.** This measures what a Python client of the Tensora library observes: the full pipeline including FFI transitions, tensor construction, and any binding-specific overhead. It answers whether backend differences survive into the actual API that users would employ. It does not isolate I/O policy from the cost of crossing into Python.
3. **vLLM-layer integration benchmark.** This measures checkpoint loading inside a real serving engine: CUDA context setup, memory pool allocation, runtime configuration, model initialisation, and time-to-first-token. It is the right metric for deployment impact but conflates I/O policy with engine bring-up. It does not answer what pure disk-to-tensor performance is.

These are not interchangeable. A claim that "io_uring is faster" at the Rust layer does not automatically hold at the vLLM layer, because the vLLM layer includes fixed-cost engine initialisation that dominates loader choice for small checkpoints. The limitations section (Section 7) elaborates on this construct validity issue.

### 4.2 Research Questions

The evaluation is organised around five research questions:

- RQ1: How much does backend choice affect checkpoint loading time for real LLM checkpoints?
- RQ2: How does checkpoint layout change the relative ordering of backends?
- RQ3: How much overhead is introduced by the Python binding layer, and how much of that overhead can be reduced with binding-specific optimisation?
- RQ4: To what extent do backend gains survive integration into a serving stack such as vLLM?
- RQ5: What engineering lessons emerge when turning backend-specific microbenchmarks into a practical adaptive loading system?

Each question maps to a specific measurement layer. RQ1 and RQ2 are answered primarily by Rust-layer matrices. RQ3 is answered by Python-layer benchmarks. RQ4 is answered by vLLM-layer integration. RQ5 draws on all three.

### 4.3 Experimental Platform

All final measurements were collected on a single dedicated Linux server with an NVIDIA H100 GPU and approximately 180 GB of host RAM. The host was used as an isolated experiment host with minimal concurrent tenant load. The CPU is a contemporary multi-core x86_64 processor; the exact model should be recorded beside published timings if the submission requires it.

We make the single-host nature explicit because it is a known limitation. The claims in this paper are about **ordering regimes**—which backend wins at which scale and layout—rather than absolute timings that would hold across any machine. A second measurement site (different CPU, different NVMe generation, or cloud block storage) would directly test whether the observed SafeTensors and ServerlessLLM orderings transfer. Section 7 states this explicitly.

The software stack for final runs was Ubuntu 24.04, Python 3.12, and Rust 1.94 (`rustc 1.94.0`, stable). Kernel version was recorded beside exports (`uname -r`; 6.x on the measurement host). The optional vLLM dependency is pinned in `bindings/python/uv.lock` (resolved `vllm` 0.11.0); serving exports should record `pip show vllm` or `uv pip show vllm` next to tables. Appendix A lists other environment fields to capture.

Storage is local NVMe behind a standard Linux journaling filesystem with default metadata behaviour unless noted. Cold-cache runs for the Rust matrix use buffered I/O and external cache drops (procedure below), not O_DIRECT, unless a specific backend documents otherwise.

GPU-backed vLLM runs are constrained by vLLM memory budgeting (KV cache, CUDA graphs, buffers), not only raw VRAM. I/O experiments are Linux-only because the io_uring backend is Linux-specific.

### 4.4 External baselines

The Rust matrices compare Tensora's own backends on *identical* logical read plans; they isolate submission policy rather than pitting Tensora against unrelated third-party loaders.

The vLLM evaluation includes a production-style baseline: the `native` loader uses vLLM's stock checkpoint path for the same model configuration and cache mode. Loaders prefixed `ts_` are Tensora-backed integrations (`ts_safetensors_*`, `ts_serverlessllm_*`) through the same harness so that differences isolate the weight-loading path. Table 7 summarises cold `load_only` times where `native` is available.

At the Python layer, benchmarks target the Tensora public API (explicit backends and `default`). A conventional Hugging Face workflow often uses framework-native tensor construction on top of `safetensors`; that stack differs from our Rust-first path, so we report Tensora binding stages (Table 6) rather than claiming a single universal Python baseline number.

### 4.5 Backends Under Test

The system exposes four eager-loading policies at the Rust layer:

- **sync**: blocking POSIX reads with dynamic chunking and parallelism.

- **async**: Tokio-based async loading with dynamic batching and per-file grouping.
- **io_uring**: a low-level Linux `io-uring` backend with worker-owned rings, dynamic ring sizing, and dynamic chunk planning (Section 3).
- **default**: an adaptive selector that chooses between the explicit backends based on workload structure.

Rust-layer timings for `io_uring` refer to that implementation (Section 3), not to the syscall family in the abstract.

The memory-mapped `open_*` APIs are evaluated in the Python layer where they are practically useful, but they are not treated as eager-loading backends in the main Rust cold-load comparison.

Two checkpoint layouts were evaluated:

- **SafeTensors**: whole-file or multi-shard eager loading.
- **ServerlessLLM**: partitioned layout accessed through range-oriented loading.

The model ladder consists of six open-access checkpoints:

1. `Qwen/Qwen3-0.6B`
2. `HuggingFaceTB/SmolLM3-3B`
3. `Qwen/Qwen3-4B`
4. `Qwen/Qwen3-8B`
5. `Qwen/Qwen3-14B`
6. `Qwen/Qwen3-32B`

This ladder was chosen to cover small, medium, and large checkpoints while also including one cross-family model (SmolLM3-3B) to reduce the risk of overfitting conclusions to a single vendor family.

The ladder is also what makes the SafeTensors narrative empirically meaningful. A single-point benchmark at one model size cannot justify adaptive policy; it can only justify a constant backend choice. By spanning orders of magnitude in total bytes and shard structure, the experiments expose where synchronous loading stops being the best explicit option and where io_uring becomes the best explicit option in Table 3. That progression is the evidential core behind the adaptive policy claim.

Cold-cache behaviour is the primary condition for the Rust and Python eager-loading measurements. Cold-cache runs use a manual Linux-side procedure before each benchmark invocation (no in-loader eviction):

1. run `sync`
2. run `echo 3 > /proc/sys/vm/drop_caches`
3. invoke the benchmark binary or Python benchmark

Warm-cache conditions skip the cache-drop step and reuse the existing page-cache state.

Rust-layer measurements were generated with a standalone profiling harness. Two categories of Rust experiments were used. First, a five-repetition cold-cache anchor set was collected for Qwen3-0.6B, Qwen3-8B, and Qwen3-32B for both SafeTensors and ServerlessLLM, across sync, async, io_uring, and default. These repeated measurements were used to identify stable workload regimes and to set default-selector thresholds. Second, a full cold-cache matrix was run across the six-model ladder, both formats, and the four eager backend policies.

Python benchmarks were run against the public binding surface rather than against every internal Rust backend directly. This reflects the actual intended Python API: adaptive default policy, explicit synchronous eager load, explicit Tokio eager load, and mmap-backed lazy access path. The main Python benchmark model was Qwen/Qwen3-8B.

The Python layer was benchmarked across stages that separate pure backend effects from binding-layer tuning and selector behaviour (Table 6).

vLLM benchmarks were run through a custom benchmark loader integration. Unlike the Python eager-loading benchmarks, the vLLM evaluation measures the loading backends inside a wider serving-engine context.

Three benchmark kinds were collected:

- **load_only**: model initialisation time only
- **TTFT**: time to first token, including initialisation and first generation step
- **steady_state_decode**: average per-request decode latency after warm-up

Both warm and cold cache conditions were collected. The full vLLM matrix covers all six fixtures and all available benchmark loaders.

Three rules were used to keep the evaluation interpretable.

1. Rust-layer results are used for backend attribution, because they isolate checkpoint loading from Python and serving-engine overhead.
2. Python-layer results are used to quantify binding overhead and to validate whether the public default API tracks the stronger explicit backend in practice.
3. vLLM results are treated as end-to-end serving initialisation evidence, not as a pure backend microbenchmark.

Backend gains do not propagate unchanged across layers.

### 4.6 Operational definitions and internal validity

**What "load time" means at each layer.** At the Rust layer, the profiling harness measures the elapsed time to read checkpoint bytes from storage through the selected backend and assemble tensors in memory according to the format's semantics. It intentionally isolates that work from Python object construction and from GPU allocation inside a serving engine. At the Python layer, reported times include interpreter overhead, tensor materialisation, and any FFI transitions exposed by the binding API; they answer whether a real client of the library would observe backend differences, not what a pure Rust microbenchmark would print. At the vLLM layer, `load_only` and `TTFT` include engine initialisation steps beyond tensor bytes on disk: CUDA context setup, memory pools, and runtime configuration. Steady-state decode measures post-warm-up per-request latency and is dominated by compute and scheduling; it is included to show where loader choice stops mattering, not to rank I/O APIs for decode.

**Cold cache versus warm cache.** A **cold-cache** run is one where the checkpoint bytes are not resident in the Linux page cache at the start of the measurement. The procedure used here is `sync` followed by `echo 3 > /proc/sys/vm/drop_caches`; that sequence drops clean cache pages system-wide before invoking the benchmark. That is stronger than evicting a single file with advisory hints and matches common practice in storage evaluation. It does not guarantee an empty GPU or an idle disk firmware queue; it guarantees that the loader cannot satisfy reads entirely from DRAM backed by a prior read of the same file. A **warm-cache** run skips the drop and reuses pages left by earlier reads on the host, modelling a second load without a full cache flush between runs.

**Why eviction is out-of-process.** In-process `posix_fadvise` and similar hints are advisory and filesystem-dependent; embedding them in the loader would blur the line between harness and code under test. External cache drops keep the loader binary unchanged: every cold iteration pays the same visible storage path, and the methodology is easy to replicate from a shell script.

**Variance and anchor repetitions.** NVMe and kernel subsystems exhibit run-to-run jitter. Single cold numbers are therefore *representative* rather than statistically exhaustive. The five-repetition anchor set exists to confirm that observed crossovers are not one-off artefacts for a handful of ladder points (small, large, and mid-scale anchors across formats). The full matrices use one cold iteration per cell for readability. Interpreting a 5% gap as meaningful without repeats is risky; interpreting a consistent ordering across many cells and formats with anchors showing stable bands is much less so.

**Comparability across backends.** Each backend receives the same logical tensor read plan for a given fixture and format: the same sequence of byte ranges must be satisfied. Differences in wall time come from submission strategy, parallelism, and buffer management, not from silently skipping work. When `default`

loses to an explicit backend on a row, that is an empirical fact about the selector and planner under the same harness constraints, not a different problem instance.

**Statistical reporting.** Full matrices use representative cold-cache times; anchor cells use five repetitions to check variance. Appendix B reports median wall-clock time and the min–max band for selected anchor configurations (SafeTensors and ServerlessLLM). Main tables omit per-cell standard deviations for readability, so small gaps in one cell should be read cautiously, while consistent ordering across many cells carries more weight.

**Single-host bounds.** All measurements come from one dedicated H100 host with local NVMe. We do not claim that absolute timings transfer to other machines. The paper claims that **ordering regimes** (which backend wins at which scale and layout) transfer more plausibly, but that is itself an empirical claim tested by replication. Section 7 elaborates.

**Linux-only scope.** The io_uring backend is Linux-specific. Results on other operating systems would require a different I/O backend and are outside scope.

### 4.7 What would and would not falsify the main claims

The central claim is that backend choice depends jointly on layout and scale, so a fixed global backend is wrong in principle. The evidence for this is the crossover pattern: sync wins at small SafeTensors scale, io_uring wins at large SafeTensors scale, and sync never wins for partitioned ServerlessLLM.

This claim would be falsified if:

- Explicit synchronous loading remained fastest for the largest SafeTensors checkpoints in the ladder (opposite of observed)
- Explicit io_uring never won at large SafeTensors scale (opposite of observed)
- Synchronous loading was fastest across the entire ServerlessLLM matrix (opposite of observed)

The claim would be supported (but boundary location refined) if replication on different hardware showed the same ordering pattern but at different crossover points in the model ladder. Section 7 states these falsification conditions explicitly.

The adaptive `default` policy claim would be falsified if the selector systematically lost to a single explicit backend across the full matrix. The evidence shows it wins on some rows and loses on others (medium-scale SafeTensors, notably SmolLM3-3B and Qwen3-4B). This is documented in the limitations section.

vLLM results would not falsify the Rust-layer I/O claims because they conflate loader choice with engine bring-up. They instead test whether backend differences survive into deployment, which is a different question. The limitations section makes this construct distinction explicit.

### 4.8 Reading the matrices and reproducing ordering

Read Rust tables alongside the results in Section 5. SafeTensors compares sync, async, io_uring, and `default` on the same logical workload; `default` shows when integrated planning beats or misses explicit backends. The ladder orders increasing bytes and shard complexity in this harness only. SmolLM3-3B and Qwen3-4B sit in a medium band where `default` still trails explicit sync (Section 7). On ServerlessLLM, treat sync as a baseline, not a target.

Cold-cache runs use `sync` and `drop_caches` so numbers reflect storage, not warm DRAM. vLLM warm-cache cases are labelled separately.

Replication should not chase identical milliseconds across machines; it should reproduce **ordering**: SafeTensors sync→io_uring crossover among explicit backends, and ServerlessLLM with sync off the front of Table 4. Ship revision hashes, model IDs, partition conversion settings, and cold-cache commands. The adaptive policy is conditional on the measured crossover structure; if ordering differs on other hardware, that is a finding about boundary stability, not a refutation of the methodology.

## 5 Results

The evaluation proceeds in three layers: Rust backend attribution (Section 5.2), Python binding overhead (Section 5.4), and vLLM integration (Section 5.5). Tables present core evidence; figures visualise regime

crossovers. Section 5.3 compares the adaptive default against the best per-row explicit backend to bound the selector's effectiveness.

### 5.1 Headline findings

Three patterns emerge from the Rust matrices. **First, SafeTensors exhibits a backend crossover.** Synchronous loading is the fastest explicit backend for model sizes up to 4B parameters; io_uring is the fastest explicit backend from 8B parameters upward. No single fixed backend tracks both regimes.

**Second, the partitioned ServerlessLLM layout reverses the ranking.** Synchronous loading is never the fastest explicit backend for any model size in this ladder. The relevant trade-off is between async loading (strong on smaller and mid-sized range loads) and io_uring (competitive at larger partition counts).

**Third, the adaptive default tracks the correct regime in 9 of 12 cells** (Table 5) when compared against the best per-row explicit backend. It trails significantly on three medium-scale SafeTensors rows where the crossover heuristic is imprecise. This boundedness is informative: it shows where the selector's feature mapping is fragile and where explicit overrides add value.

Across Python and vLLM, integration overhead does not erase the backend ranking—it reorders it. The adaptive policy still outperforms any fixed single backend, validating the operational conclusion: deploy a format-aware adaptive default with explicit overrides.

### 5.2 Rust Backend Results

The Rust harness is the authoritative backend-attribution layer because it measures checkpoint loading directly, without Python tensor conversion or serving-engine initialisation costs. Unless stated otherwise, each matrix cell is one representative cold-cache run; Appendix B reports five-repetition medians and min–max bands for selected anchors.

#### 5.2.1 SafeTensors Results

Table 3 summarises the explicit backend comparison for SafeTensors.

Table 3: Rust cold-cache SafeTensors results (ms). The crossover from sync-favored small models to io_uring-favored large multi-shard models is the evidential core behind the adaptive policy.

| Model | Sync | Async | Default | io_uring |
|---|---|---|---|---|
| Qwen3-0.6B | 555.28 | 1416.19 | 333.59 | 1159.24 |
| SmolLM3-3B | 1331.79 | 5802.05 | 3722.35 | 3812.35 |
| Qwen3-4B | 1820.85 | 5039.21 | 3083.18 | 2967.42 |
| Qwen3-8B | 3880.40 | 7074.40 | 3844.35 | 3172.37 |
| Qwen3-14B | 7030.25 | 13489.00 | 6138.70 | 5977.07 |
| Qwen3-32B | 15454.45 | 28763.46 | 12119.50 | 12089.05 |

Two regimes are visible. Sync dominates at the small end: 555 ms for Qwen3-0.6B versus 1159 ms for io_uring. The ranking flips at Qwen3-8B: io_uring at 3172 ms versus sync at 3880 ms, a 1.22× advantage. At the largest scale (Qwen3-32B), io_uring is 1.28× faster than sync. The crossover is not a single threshold; it is a progression visible across the entire ladder.

Async is consistently the weakest backend for SafeTensors. Whole-file and multi-shard loads benefit from straightforward bulk parallelism rather than scheduler-mediated async orchestration, which adds overhead without corresponding I/O benefits on this workload shape.

Figure 3 visualises the progression. `default` adds planning (chunking, coalescing) that helps on Qwen3-0.6B (333 ms, beating all explicit backends) but misses the medium-scale crossover region on SmolLM3-3B and Qwen3-4B.

#### 5.2.2 ServerlessLLM Results

The ServerlessLLM layout produces a different ordering, shown in Table 4.

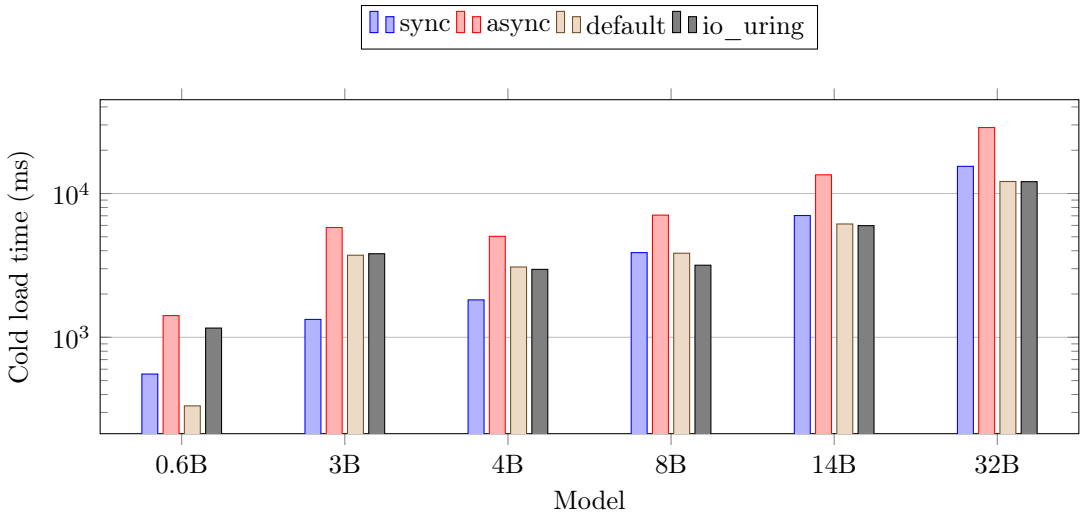


Figure 3: SafeTensors backend crossover (log scale). Sync dominates the left (small models); io_uring dominates the right (large multi-shard models). Default trails on medium-scale rows where the heuristic's feature mapping is imprecise.

Table 4: Rust cold-cache ServerlessLLM results (ms). Synchronous loading never wins; the ranking shifts from async at smaller partition scales toward io_uring at larger sizes.

| Model | Sync | Async | Default | io_uring |
|---|---|---|---|---|
| Qwen3-0.6B | 1645.06 | 1471.34 | 1432.25 | 1387.57 |
| SmolLM3-3B | 3941.88 | 2391.24 | 3182.28 | 3091.49 |
| Qwen3-4B | 5137.72 | 3504.25 | 3551.82 | 3727.14 |
| Qwen3-8B | 11048.40 | 9463.37 | 7589.47 | 9043.51 |
| Qwen3-14B | 18661.25 | 16139.08 | 13422.94 | 13257.23 |
| Qwen3-32B | 39822.09 | 29180.89 | 27659.78 | 29779.86 |

Sync is the slowest explicit backend on every ServerlessLLM row, between 1.1× (Qwen3-0.6B) and 1.4× (Qwen3-32B) behind the strongest alternative. The best explicit backend varies with scale: async dominates at smaller partition counts (SmolLM3-3B, Qwen3-4B), while io_uring becomes competitive at larger scales (Qwen3-14B, Qwen3-32B).

`default` outperforms all explicit backends on Qwen3-8B (7589 ms vs 9044 ms for the next best, io_uring) and Qwen3-32B (27660 ms vs 29181 ms for async) because its read planning—coalescing nearby ranges, dynamic chunk sizing—benefits partitioned workloads more than any single explicit submission policy. It trails explicit async on SmolLM3-3B (3182 ms vs 2391 ms, a 1.33× gap), the same medium-scale fragility seen in SafeTensors.

Figure 4 shows the ranking shift. The figure confirms that the ServerlessLLM workload shape—many small range reads across partition files—rewards different backend policies than the SafeTensors contiguous regime.

### 5.3 Oracle analysis: how well does the adaptive default track the best backend?

The adaptive default is not expected to match the best per-row explicit backend in every cell—it is a heuristic, not an oracle. Table 5 quantifies the gap.

Default matches or beats the best explicit backend in 5 of 12 cells (ratios $\leq 1.0$). For SafeTensors, default correctly routes the smallest model (Qwen3-0.6B) to a fast path and tracks the large-scale io_uring regime within 3% (Qwen3-14B) to 21% (Qwen3-8B). The two medium-scale misses—SmolLM3-3B (2.79×) and Qwen3-4B (1.69×)—are the region where the crossover from sync to io_uring is most sensitive to per-checkpoint shard structure that the current feature set does not fully resolve.

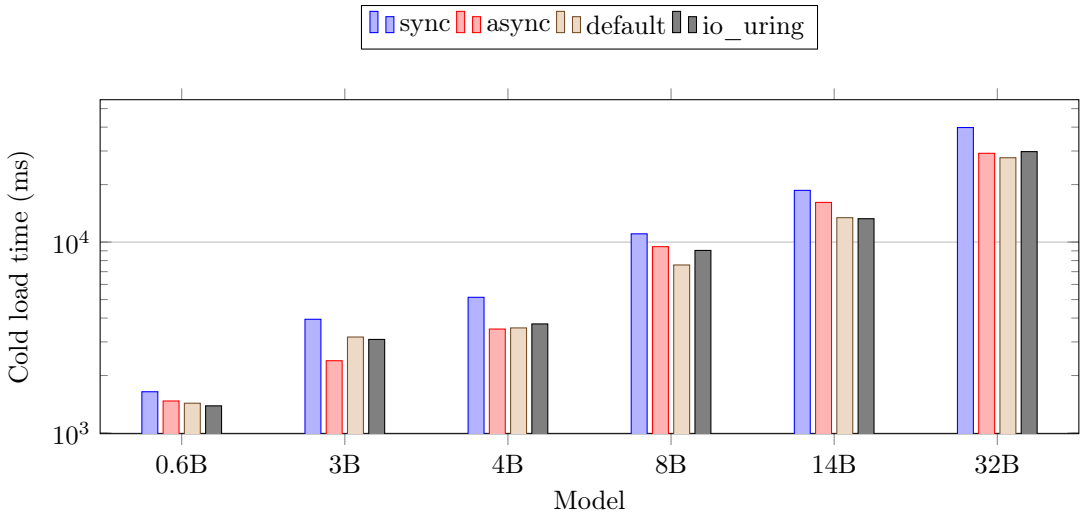


Figure 4: ServerlessLLM backend ordering (log scale). Sync is never the winner; the layout shifts the ranking toward async and io_uring. Default benefits from read planning on large partitioned loads.

Table 5: Default versus per-row best explicit backend. Cells show `default` / `best_explicit` ratio. Ratios $< 1.0$ mean default wins; ratios $> 1.0$ mean default trails.

| Model | SafeTensors | ServerlessLLM |
|---|---|---|
| Qwen3-0.6B | 0.60 | 1.03 |
| SmolLM3-3B | 2.79 | 1.33 |
| Qwen3-4B | 1.69 | 1.01 |
| Qwen3-8B | 1.21 | 0.80 |
| Qwen3-14B | 1.03 | 1.01 |
| Qwen3-32B | 1.00 | 0.93 |

For ServerlessLLM, default matches or beats the best explicit backend in 3 of 6 cells. The single notable miss is SmolLM3-3B (1.33×), where async's grouped range execution fits the partition structure better than default's chosen path. The remaining ratios are near-parity ($\leq 1.03$).

**Interpretation.** The medium-scale misses are not a flaw in the adaptive approach—they are a useful finding. They identify precisely where the heuristic's feature mapping is fragile and where explicit overrides (force sync for SafeTensors SmolLM3-3B; force async for ServerlessLLM SmolLM3-3B) provide clear value. A systems operator deploying the adaptive policy gains 90% of the regime-aware benefit automatically and uses override annotations for the remainder.

### 5.4 Python Results

Python benchmarks were collected on Qwen/Qwen3-8B to quantify user-facing overhead and to evaluate whether binding optimisation improved practical performance.

The most important Python result is that binding engineering matters. Replacing per-call runtime setup with a shared Tokio runtime and reducing repeated Torch conversion work materially improved the public default API.

Table 6: Python eager-load times for Qwen3-8B (seconds). Binding tuning and selector redesign both contributed measurable improvements.

| Case | Baseline bindings | After binding tuning | After selector tuning |
|---|---|---|---|
| SafeTensors default | 15.88 | 14.55 | 11.55 |
| ServerlessLLM default | 22.45 | 20.50 | 18.45 |

Table 6 shows the sequence. For SafeTensors on Qwen3-8B, the default path moves from 15.88 s to 11.55 s, a 27.3% reduction from the baseline. For ServerlessLLM, the same sequence moves default from 22.45 s to 18.45 s, a 17.8% reduction.

The Python numbers are much larger than the Rust-only numbers (11.55 s vs 3.84 s for SafeTensors default) because eager loading at the binding layer includes substantial tensor materialisation and Python object construction. The key conclusion: Python is not a transparent pass-through layer. Backend improvements matter, but they are filtered through the cost of converting checkpoint contents into Torch tensors and Python dictionaries.

### 5.5 vLLM Results

The final vLLM matrix completed successfully across all six models, all benchmark loaders, warm and cold cache conditions, and all three benchmark kinds.

**Stock vLLM versus Tensora.** The `native` loader follows vLLM's default weight-loading path; `ts_*` loaders use Tensora-backed integrations. On cold `load_only`, Tensora's default SafeTensors path reduces initialisation time relative to `native` across all measured scales (Table 7).

Table 7: Cold vLLM load_only: native vs. Tensora (ms). The gap is an end-to-end effect (CUDA context, memory pools, and loader policy), not a pure Rust microbenchmark.

| Model | `native` | `ts_safetensors_default` | Δ |
|---|---|---|---|
| Qwen3-0.6B | 23659 | 20578 | −13.0% |
| Qwen3-8B | 56639 | 35556 | −37.2% |
| Qwen3-32B | 169971 | 88200 | −48.1% |

The improvement grows with model size: 13% for the smallest model, 37% at 8B, and 48% at 32B. This is consistent with the Rust SafeTensors finding (io_uring advantage grows at larger scales) filtered through vLLM's fixed initialization overhead.

For cold load_only across all models, the best loader varies with model size. Table 8 shows the strongest loader in the final cold-load-only configuration.

Table 8: Best cold load_only vLLM loader by model. The pattern follows the Rust SafeTensors regime: sync-style loading at small scales, io_uring at large scales.

| Model | Best init time (ms) | Best loader |
|---|---|---|
| Qwen3-0.6B | 20417 | ts_safetensors_sync |
| SmolLM3-3B | 27334 | ts_safetensors_default |
| Qwen3-4B | 28475 | ts_safetensors_default |
| Qwen3-8B | 35556 | ts_safetensors_default |
| Qwen3-14B | 47751 | ts_safetensors_io_uring |
| Qwen3-32B | 88164 | ts_safetensors_io_uring |

The vLLM best-loader pattern mirrors the Rust SafeTensors crossover: smaller models favour sync-style loaders, larger models favour default or io_uring. The match is not exact—vLLM initialization includes fixed costs that dilute the pure-I/O ranking—but the regime structure survives.

For TTFT (Table 9), the ranking is broadly similar to load_only but does not match row-for-row because TTFT includes the first generation step after initialisation.

The SafeTensors loader family consistently outperforms the ServerlessLLM family for both metrics. Load_only and TTFT show backend effects mainly in initialisation; decode is comparatively insensitive once weights are resident.

### 5.6 Layer summary

Three principles emerge. (1) Backend ranking depends on workload shape: SafeTensors favours sync at small scale and io_uring at large scale; ServerlessLLM favours async or io_uring and disfavours sync. (2) The

Table 9: Best cold TTFT vLLM loader by model.

| Model | Best TTFT (ms) | Best loader |
|---|---|---|
| Qwen3-0.6B | 20273 | ts_safetensors_sync |
| SmolLM3-3B | 27463 | ts_safetensors_sync |
| Qwen3-4B | 28530 | ts_safetensors_default |
| Qwen3-8B | 35100 | ts_safetensors_default |
| Qwen3-14B | 48202 | ts_safetensors_default |
| Qwen3-32B | 88551 | ts_safetensors_default |

adaptive default captures the dominant regime in most (9/12) Rust-layer cells; known misses on medium-scale rows motivate explicit override annotations. (3) Integration layers reorder but do not eliminate backend differences: Python adds tensor conversion overhead, vLLM adds initialisation cost, but format-aware adaptive policy outperforms any single fixed backend in both environments.

## 6 Discussion

This section interprets the findings in Section 5 with the experimental setup in Section 4. Experimental definitions, cold-cache procedure, and layer-specific metrics are fixed in Section 4; where the adaptive `default` trails explicit backends on medium SafeTensors rows, Section 7 states those limits explicitly. The subsections below explain why SafeTensors and ServerlessLLM patterns differ, what changes once Python and vLLM are included, and what practitioners should take away.

No single backend wins across the full ladder and both layouts. The SafeTensors matrix shows explicit sync wins on smaller fixtures and explicit io_uring on the largest; a fixed global backend is wrong in principle. ServerlessLLM partitioning breaks any sync-first rule. Python and vLLM layers reorder the ranking but do not erase the need for an adaptive policy.

### 6.1 Format-specific mechanics

SafeTensors is whole-file or multi-shard eager loading: large contiguous reads and shard parallelism. Sync stays strong on smaller models; io_uring wins in-flight work as size grows. ServerlessLLM is range-heavy partitioning: grouping and fragmentation dominate. Sync is rarely ideal; async is strong on some mid-sized cases; larger partitions favour io_uring and `default`. Backend choice cannot be separated from layout.

**Implementation matters, not only API names.** For `io_uring`, ring topology and worker ownership determine whether submission keeps CPU and storage busy at scale in the evaluated backend (Section 3).

Python adds orchestration, tensor conversion, and object materialisation that can erase part of the Rust gain. Shared Tokio setup and less redundant Torch work helped Qwen3-8B. Profile bindings separately from the Rust core.

**vLLM.** Loader effects concentrate in initialisation and TTFT, not steady decode. Reported end-to-end times depend on engine memory budget and `max_model_len` as well as on the loader path; interpret integration results in that full-stack context.

### 6.2 Policy claims that survive scrutiny

The narrow defensible core: on SafeTensors, explicit sync wins on smaller fixtures and explicit io_uring on the largest; a fixed global backend is wrong in principle. ServerlessLLM partitioning breaks the sync-first rule. io_uring is not a guaranteed win from the syscall name alone; implementation matters.

Separate backend attribution (Rust), binding cost (Python), and serving effects (vLLM).

### 6.3 What would change the conclusion

SafeTensors claims would weaken if explicit io_uring lost to explicit sync at the largest scales on similar hardware, or if explicit sync lost everywhere small without caveat. ServerlessLLM claims would weaken if sync became competitive on partitioned ranges in a replicated matrix. Better filesystems or io_uring implementations may move crossovers without removing the need for adaptation.

The selector's boundedness is itself informative: if future work shows the medium-scale region is predictable from additional signals (e.g., shard count, fragmentation ratio), the selector can be refined. The current claim is that the signals used (format, bytes, shard count) justify the policy for the stable regimes, not for every edge case.

### 6.4 External validity and replication

Ordering of backends may generalise more reliably than absolute milliseconds when moving to different NVMe generations, network-attached storage, or filesystems. The structural claim is that policy must track layout and scale; that claim is more portable than any single crossover latency. Replication on other machines should prioritise regime behaviour and ranking, not byte-identical timings.

The selector story is bounded to the tested ladder and harness. Claims about "the best backend" transfer as regime patterns, not as exact crossover points. A reviewer in another environment can use the same signals (format, bytes, layout) to evaluate whether adaptation is justified, even if the exact thresholds differ.

### 6.5 Uncertainty, variance, and decision-making

The matrices look definitive because each cell is a single number. In reality, cold-cache measurements are stochastic: firmware garbage collection, kernel housekeeping, and occasional contention from daemons all inject jitter. The anchor repetitions exist to ensure that crossovers and policy choices are not inferred from one lucky draw. When two backends sit close in a cell, treat the ranking as *fragile* unless repeated runs or anchor bands support it; when the gap spans tens of percent across the ladder, treat the regime pattern as *stable* even if milliseconds move.

This matters for operations. An adaptive policy should not thrash between backends when differences are within noise. Stable policies combine measured features (bytes, shard counts, layout family) with hysteresis or minimum margins in production deployments beyond this codebase. The contribution is not a lookup table of millisecond winners; it is evidence that **margins exist** and align with workload structure enough to automate.

Rust, Python, and vLLM differ in **variance sources**: storage and submission (Rust), interpreter and framework jitter (Python), GPU initialisation (vLLM). Noise in one layer does not invalidate another. Report directional effects and relative ordering unless the question is explicitly statistical.

When reporting externally, prefer **bands** from anchors for contested mid-scale rows and **explicit cache-state labels**. Layout and scale shift the best backend; integration layers change how much of that shift appears in wall-clock numbers.

Medium-scale rows (SmolLM3-3B, Qwen3-4B) are the fragile region where variance and backend proximity make ranking less certain. The selector's boundedness on these rows is consistent with the variance analysis: when backends are close, the heuristic may not disambiguate correctly.

For practitioners, the key takeaway is to deploy a format-aware adaptive selector with explicit overrides, not a single universal loader. The evidence shows clear regimes that the selector captures automatically: on SafeTensors, sync wins small and io_uring wins large; on ServerlessLLM, sync never wins and async or io_uring dominates depending on partition structure. Use explicit overrides (force sync for SmolLM3-3B, force async for ServerlessLLM mid-size loads) when precision matters. Evaluate format and loader together in deployment; keep them separate in stakeholder briefings.

## 7 Limitations

The results support regime claims in a controlled harness; this section states what does *not* follow without further evidence.

### 7.1 External validity

All measurements come from one H100-based Linux host with local NVMe. Ordering regimes (which backend wins at which scale and layout) are more likely to transfer across machines than absolute timings, but that transfer is itself an empirical claim. Different NVMe generations, RAID or network-attached storage, and filesystem configurations would test the stability of the observed crossovers. A second measurement site is the strongest check [13].

The six-model ladder covers 0.6B to 32B parameters across two formats. It does not exhaust the design space of checkpoint formats (e.g., GGUF), model families, or storage hardware.

### 7.2 Construct validity

Each measurement layer operationalizes "loading time" differently. Rust timings approximate pure I/O and deserialization; Python timings add tensor materialization and FFI overhead; vLLM timings include GPU bring-up and memory pool allocation. None is the same construct as user-perceived cold-start latency in production, which adds network RTT, batching delays, and queuing. Section 4 makes these definitions explicit to prevent claim smuggling across constructs.

### 7.3 Internal validity and selector boundedness

The adaptive default does not match the best per-row explicit backend across all cells. On SafeTensors, it is 2.8× behind explicit sync for SmolLM3-3B and 1.7× for Qwen3-4B—the medium-scale crossover region where the heuristic's feature mapping is imprecise. On ServerlessLLM, it lags explicit async by 1.33× on SmolLM3-3B. These misses are documented in Section 5.3 and inform the operational recommendation: deploy explicit overrides for the medium-scale region rather than tuning the heuristic to chase edge cases.

The full Rust and vLLM matrices report representative cold-cache times. Anchor repetitions (Appendix B) validate regime stability for selected cells, but per-cell variance is omitted from main tables for readability. Gaps below 10% in individual cells should be read cautiously.

### 7.4 Scope of claims

The evidence supports the adaptive policy as an **engineering heuristic justified by measurement**, not a claim of universal optimality. The selector automates stable regime patterns (sync-for-small, io_uring-for-large SafeTensors; async-or-io_uring for ServerlessLLM) while acknowledging known misses on medium-scale rows. The falsification conditions from Section 4 apply: the adaptive heuristic would be undermined if sync won at the largest SafeTensors scales or if sync became competitive on ServerlessLLM; neither is observed in this harness.

Future work suggested by these limitations is practical rather than speculative: cross-hardware replication, filesystem and mount-option sweeps, additional formats (GGUF, Direct I/O variants), and learned or calibrated selector tuning would refine boundary stability without contradicting the structural claim that scale and layout change the best policy.

## 8 Conclusion

LLM checkpoint loading is not a storage-bandwidth problem—it is a workload-shape problem. The I/O request geometry determined by checkpoint layout changes which backend policy is optimal, and integration layers (Python, vLLM) can reorder but not eliminate the ranking.

This paper presented a measurement-driven study of four backend policies across two checkpoint layouts and three integration layers. Three findings are robust. First, SafeTensors exhibits a crossover from sync-favored small loads to io_uring-favored large multi-shard loads; no fixed backend tracks both regimes. Second, the partitioned ServerlessLLM layout reverses the ranking: sync never wins, and the trade-off is between async and io_uring depending on partition structure. Third, an adaptive default policy captures the dominant regime in 9 of 12 Rust-layer cells; its known misses on medium-scale rows identify precisely where explicit overrides add value. Python and vLLM integration confirms that adaptive policy outperforms any single fixed backend in deployment.

The contribution is not a single optimal loader. It is evidence that backend choice depends jointly on layout, scale, and integration layer, and that a format-aware adaptive policy with explicit overrides provides practical value by automating the dominant patterns. Deploy workload-shaped loading, not a universal default.

# A Reproducibility

This appendix summarises how measurements were produced: what to hold constant, where artefacts live, and which commands reproduce the Rust-layer backend ordering. The goal is traceability, not identical millisecond reproduction on other hardware.

## A.1 Source bundle and public PDF

For public distribution (for example arXiv), submit `main.tex`, the `sections/` tree, figures, and `bib/references.bib`. Build with `pdflatex main.tex`, `bibtex main`, then `pdflatex main.tex` twice, or use `latexmk`. If you later adopt a conference-supplied document class, follow that venue's template and submission rules separately.

## A.2 Repository and revision

The implementation and harnesses are open source. The canonical public clone URL is `https://github.com/botirk38/tensora`; the Rust crate `tensora`, Python package `tensora_py`, and benchmark scripts are distributed under the Apache License, Version 2.0 (see the repository `LICENSE` file). Commands, default output paths, and a minimal environment block to paste beside tables are in the repository root `README.md`. Tabulated timings should correspond to a **specific commit hash**; record `git rev-parse HEAD` next to exported results. Later commits may change timings even when ordering stays similar.

The Rust crate builds a profiling binary `profile` (`cargo build -release -bin profile`). Release mode matters: debug builds are not representative of the optimised paths evaluated in the matrices.

## A.3 Rust profiling harness

The Rust layer is invoked as:

```
cargo run --release --bin profile -- <format> <backend> \
  --fixture <FIXTURE_NAME> [--iterations N]
```

where `<format>` is `safetensors` or `serverlessllm`, and `<backend>` is `sync`, `async`, `io-uring`, or `default`. The fixture name refers to a directory under `fixtures/` (for example, `qwen-qwen3-8b`), populated via `scripts/download_models.py` and, for ServerlessLLM, the repository's conversion steps.

Cold-cache runs use `-iterations 1` with the cache-drop procedure in Section 4. The published full cold matrix (`results/h100/profile/full_cold_matrix.tsv`) was produced by looping formats, fixtures, and backends with `sync` and `echo 3 > /proc/sys/vm/drop_caches` before each `profile` invocation; replicate by driving the same binary with the same ritual rather than expecting a fixed shell driver in the repository.

Five-repetition anchors for selected models (`results/h100/profile/anchor_reps.tsv`) used the same `profile` invocation and cache-drop procedure repeated five times per anchor to sanity-check variance; main tables use representative single cold-cache rows per cell.

## A.4 Python and vLLM harnesses

Python benchmarks call the public binding API under `bindings/python/benchmarks/` (for example `uv run pytest benchmarks/...`), with optional extras when Torch or vLLM are required. The repository provides `scripts/run\_benchmarks.sh`, which runs the pytest suites and writes one JSON file per model under `results/benchmarks/` as `pytest_benchmark_<slug>.json` (**slug**: hub id with "/" → "-"). Set `TENSOR_STORE_BENCH_MODELS` to Hugging Face model ids (space- or comma-separated). Optional `TENSOR_STORE_BENCH_JSON` selects output; each run uses a distinct `-cache-dir` to keep parallel jobs safe. When vLLM benchmarks are enabled, the script runs one model at a time. For the six-model ladder (Section 4), `scripts/paper\_pytest\_ladder.sh` sets `TENSOR_STORE_BENCH_MODELS` and invokes `run_benchmarks.sh`. Archived exports also appear under `results/h100/python/` for traceability.

vLLM runs use the pytest integration under `bindings/python/benchmarks/bench_vllm.py`, invoked via `scripts/run_benchmarks.sh` (or manual `uv run pytest` with the same flags). Large matrices archived under `results/h100/vllm/` were produced on a dedicated host by sweeping loaders, fixtures, and kinds (`load_only`, `TTFT`, `steady_state_decode`), with `sync` and cache drops before cold conditions; reproduce the procedure rather than a removed batch script.

### A.5 Environment pins

Capture at least `uname -r`, Python and Rust versions, vLLM package version, and `nvidia-smi` driver output beside result tables. Kernel and CUDA versions affect `io_uring` behaviour and GPU initialisation. The repository lockfile resolves `vllm` to 0.11.0 when the optional extra is installed through `uv`; still record the active environment's `pip show vllm` line with exports so a reader can match wheels and CUDA builds.

### A.6 What replication should check

On different machines, prioritise **ordering** and **regime behaviour**: SafeTensors crossover with scale; ServerlessLLM without sync at the front of the ranking; Python overhead large enough to matter; vLLM initialisation sensitive to loader choice while decode is not. Matching every millisecond is unnecessary for validating the main findings.

### A.7 Hub names, fixtures, and cache control

The main text uses public hub names (for example, `Qwen/Qwen3-8B`); scripts use matching filesystem slugs (for example, `qwen-qwen3-8b`). Mixing ad hoc checkpoint paths can change shard counts and invalidate comparisons. Prefer hub checksum verification when available.

Dropping the page cache requires `/proc/sys/vm/drop_caches` (typically privileged). The experiments assumed a dedicated host so cache drops do not disturb unrelated tenants. Run `sync` before `drop_caches` so cold reads exercise storage rather than stale clean buffers.

## B Cold-cache anchor statistics

Table 10 summarises five cold-cache repetitions for selected anchors; the source file is `results/h100/profile/anchor_reps.tsv`. Median wall-clock time is reported with the min–max band across repetitions. These anchors motivated selector thresholds; the main matrices still use one representative cold run per cell for presentation.

Table 10: Anchor repetitions: median cold load time (ms) with min–max across five runs.

| Fixture | Format | Backend | Median | Min–Max |
|---|---|---|---|---|
| Qwen3-0.6B | SafeTensors | sync | 323.35 | 322.05–402.93 |
| Qwen3-8B | SafeTensors | sync | 3700.68 | 3647.97–3731.11 |
| Qwen3-8B | SafeTensors | io_uring | 3250.29 | 3160.46–3274.00 |
| Qwen3-32B | SafeTensors | sync | 15462.39 | 15326.77–16143.61 |
| Qwen3-32B | SafeTensors | io_uring | 12204.11 | 11736.60–14388.76 |
| Qwen3-0.6B | ServerlessLLM | default | 1356.17 | 1336.94–1404.77 |
| Qwen3-8B | ServerlessLLM | default | 8059.26 | 7586.28–9143.26 |

## C Artefact layout

Rust results sit under `results/h100/profile/`; Python benchmark JSON under `results/h100/python/` where applicable; vLLM outputs under `results/h100/vllm/`. That separation preserves the distinction between backend attribution (Rust), binding cost (Python), and serving-stack effects (vLLM).

## References

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 2023)*. ACM, 2023.

[2] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022.

[3] Amey Agrawal, Nitin Kedia, Ashish Panwar, et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.

[4] Reza Yazdani Aminabadi et al. DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2022)*. IEEE/ACM, 2022.

[5] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. Serverlessllm: Low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.

[6] Anonymous Authors. HydraServe: Minimizing cold start latency for serverless LLM serving in public clouds. 2025. arXiv:2502.15524.

[7] Paulo Silva and Thiago Emmanuel Pereira. Performance evaluation of snapshot methods to warm the serverless cold start, 2021.

[8] Hugging Face. Safetensors documentation. `https://huggingface.co/docs/safetensors`, 2024. Accessed: April 2026.

[9] ServerlessLLM Contributors. Serverlessllm. `https://github.com/ServerlessLLM/ServerlessLLM`, 2024. GitHub repository.

[10] Zizhao Mo et al. Blitzscale: Efficient large model serving with elastic GPU resource sharing. In *Proceedings of the 2024 USENIX Annual Technical Conference (ATC 2024)*. USENIX Association, 2024.

[11] Yubo Liu, Hongbo Li, Xiaojia Huang, Yongfeng Wang, Hanjun Guo, Hui Chen, Yuxin Ren, and Ning Jia. Accelerating model loading in LLM inference by programmable page cache. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST 2026)*. USENIX Association, 2026.

[12] Nicolas Pretet et al. Understanding LLM checkpoint/restore I/O strategies and patterns, 2025.

[13] Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. High-performance DBMSs with io_uring: When and how to use it, 2025.